

module-08-files

Chapter 8: Reading and Writing Files | 40 programs in learning order

Source: Automate the Boring Stuff with Python 1e, Al Sweigart (CC-BY-NC-SA). Syllabus Ch8: paths, cwd, makedirs, absolute/relative, sizes, open/read/write, shelve, pprint, quiz + clipboard + mad-libs

How to use this guide (beginner path)

1) Read Concepts once. 2) Study the Visuals - redraw each on paper. 3) Follow the Program map. 4) Work programs 01..N in order. [shell]: type every plain line into IDLE, compare with # OUT: comments (errors shown are intentional demos). [script]: save, predict output, run with F5. 5) Drill Interview prep. 6) Teach it: explain each visual aloud without looking.

Concepts in detail

Files live at PATHS. `os.getcwd()/os.chdir()` show/change the working folder; `os.makedirs()` builds nested folders; `os.path.join()` builds correct paths per OS; `abspath()/relpath()` translate; `getsize()/listdir()` inspect; `exists()/isfile()/isdir()` validate BEFORE you open. `open() + read()/readlines()/write()` move text; close when done (or use `with`). `shelve` stores Python values like a mini-database; `pprint.pformat()` saves readable snapshots.

Key ideas to carry forward

- 'C:\\spam' needs doubled backslashes; forward slashes also work on Windows.
- Relative 'data/f.txt' vs absolute 'C:/data/f.txt' - know which you hold.
- Quiz generator (randomized tests), `multiclipboard` (`shelve`), and `mad libs` are the applied builds.

Classic beginner mistakes

- ! Opening a file in the wrong folder (cwd confusion) - print `os.getcwd()` first.
- ! Forgetting to close/write-mode 'w' erasing a file you meant to read.

Study order hint: 01-16 paths and inspection, 17-27 open/read/write/shelve, 28-40 the three projects in order.

Interview prep - questions you must answer cold

Say each answer aloud in under 60 seconds, then prove it with one program from this module.

Q1. Absolute vs relative path?

A: Absolute starts at root (C:/data/f.txt); relative starts at cwd (data/f.txt). Scripts break when cwd differs - print `os.getcwd()`.

Q2. How do you build portable paths?

A: `os.path.join('a','b')` uses the right separator per OS; never hard-code slashes.

Q3. Check paths before opening?

A: `exists()/isfile()/isdir()` validate; `getsize()/listdir()` inspect. Prevents crashes on typos.

Q4. Text read/write skeleton?

A: with `open(p)` as `f`: `data = f.read()` auto-closes. 'w' overwrites, 'a' appends, 'r' reads.

Q5. shelve vs plain text?

A: shelve persists Python objects (dict-like) across runs - ideal for multiclipboard; text files suit human-readable content.

Q6. pprint.pformat use?

A: Dumps structures as readable Python text you can save and later import back.

Teach it to others

The one analogy that sticks

The filesystem is a building: cwd is the room you stand in, relative paths are directions from here ('next door'), absolute paths are full addresses. open() modes are door signs: r look, w repaint (erases!), a add a poster.

What your students will get wrong

- x 'w' adds to a file (it truncates - use 'a').
- x The script's folder equals cwd (it equals where you launched).
- x Backslash paths work raw ('\n' becomes newline - double them or use /).

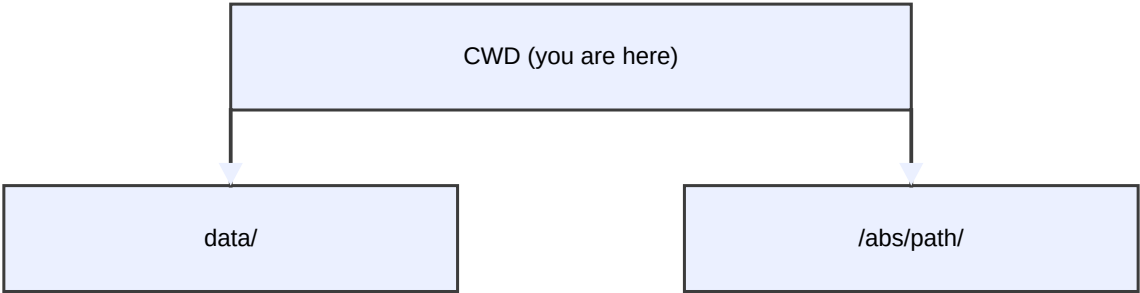
5-minute whiteboard plan

1. Map os.getcwd()/chdir/makedirs on a drawn tree.
2. Crash-then-guard: open missing file, add exists() check.
3. Students write + read back a note with with-blocks.
4. Exit ticket: join() a portable path for their OS.

Visualize it - draw these from memory

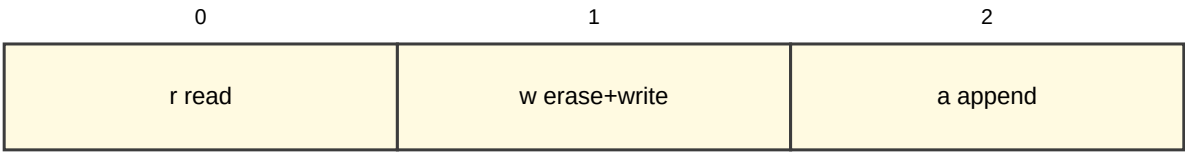
Redraw each figure on paper with labels before you touch code. If you cannot draw it, you cannot teach it.

Visual 1 - you stand in cwd; paths are directions



Relative = from here (data/f.txt). Absolute = full address. Scripts break when launched elsewhere - print getcwd().

Visual 2 - door signs: r look, w repaint, a poster



'w' truncates! Open existing work with 'r' or 'a'. with-blocks auto-close the door.

More examples - same idea, new clothes

Three bonus drills per module. Cover the output, predict, then run.

X1. Safe note saver

```
import os
p = 'note.txt'
mode = 'a' if os.path.exists(p) else 'w'
with open(p, mode) as f:
    f.write('hello\n')
print(os.path.getsize(p), 'bytes')
# EXPECTED: Appends hello, prints size
```

X2. List big files

```
import os
big = [f for f in os.listdir('.') if os.path.isfile(f) and os.path.getsize(f) > 1000]
print(len(big), 'files over 1KB')
# EXPECTED: e.g. 3 files over 1KB
```

X3. Config round-trip

```
from pprint import pformat
cfg = {'theme': 'dark', 'size': 12}
with open('cfg.py', 'w') as f:
    f.write('CFG = ' + pformat(cfg))
print('saved')
# EXPECTED: saved (CFG = {...} in cfg.py)
```

Program map - all 40 at a glance

```
01 01_backslash_windows_forward.py
02 02_backslash_windows_forward_myfiles.py
03 03_current_working_directory.py
04 04_current_working_directory_os.py
05 05_creating_new_folders.py
06 06_handling_absolute_relative.py
07 07_handling_absolute_relative_os.py
08 08_handling_absolute_relative_path.py
09 09_handling_absolute_relative_calcfilepath.py
10 10_handling_absolute_relative_os_2.py
11 11_handling_absolute_relative_calcfilepath_2.py
12 12_handling_absolute_relative_usr.py
13 13_finding_file_sizes.py
14 14_finding_file_sizes_totalsize.py
15 15_checking_path_validity.py
16 16_checking_path_validity_os.py
17 17_opening_files_open.py
18 18_opening_files_open_hellofile.py
19 19_reading_contents_files.py
20 20_reading_contents_files_when.py
21 21_reading_contents_files_sonnetfile.py
22 22_writing_files.py
23 23_saving_variables_shelve.py
24 24_saving_variables_shelve_shelffile.py
25 25_saving_variables_shelve_shelffile_2.py
26 26_saving_variables_pprint.py
27 27_saving_variables_pprint_mycats.py
28 28_step_1_store.py
29 29_step_2_create.py
30 30_step_3_create.py
31 31_step_4_write.py
32 32_step_4_write_name.py
33 33_step_4_write_d.py
```

34 34_project_multiclipboard.py
35 35_step_1_comments.py
36 36_step_2_save.py
37 37_step_3_list.py
38 38_mad_libs.py
39 39_mad_libs_enter.py
40 40_mad_libs_the.py

Topic: Backslash on Windows and Forward Slash on OS X and Linux

01/40 01_backslash_windows_forward.py [shell]

Interactive: type each plain line into IDLE; # OUT: lines are expected results.

Interactive session from the book. Run line-by-line in IDLE.

Lines starting with >>> are input; other lines are expected output (commented).

```
import os
os.path.join('usr', 'bin', 'spam')
# OUT: 'usr\\bin\\spam'
```

02/40 02_backslash_windows_forward_myfiles.py [shell]

Interactive: type each plain line into IDLE; # OUT: lines are expected results.

```
# Interactive session from the book. Run line-by-line in IDLE.  
# Lines starting with >>> are input; other lines are expected output (commented).  
  
myFiles = ['accounts.txt', 'details.csv', 'invite.docx']  
for filename in myFiles:  
# OUT:         print(os.path.join('C:\\Users\\asweigart', filename))  
# OUT: C:\\Users\\asweigart\\accounts.txt  
# OUT: C:\\Users\\asweigart\\details.csv  
# OUT: C:\\Users\\asweigart\\invite.docx
```

Topic: The Current Working Directory

03/40 03_current_working_directory.py [shell]

Interactive: type each plain line into IDLE; # OUT: lines are expected results.

Interactive session from the book. Run line-by-line in IDLE.

Lines starting with >>> are input; other lines are expected output (commented).

```
import os
os.getcwd()
# OUT: 'C:\\Python34'
os.chdir('C:\\Windows\\System32')
os.getcwd()
# OUT: 'C:\\Windows\\System32'
```

04/40 04_current_working_directory_os.py [shell]

Interactive: type each plain line into IDLE; # OUT: lines are expected results.

```
# Interactive session from the book. Run line-by-line in IDLE.  
# Lines starting with >>> are input; other lines are expected output (commented).  
  
os.chdir('C:\\ThisFolderDoesNotExist')  
# OUT: Traceback (most recent call last):  
# OUT:   File "<pyshell#18>", line 1, in <module>  
# OUT:     os.chdir('C:\\ThisFolderDoesNotExist')  
# OUT: FileNotFoundError: [WinError 2] The system cannot find the file specified:  
# OUT: 'C:\\ThisFolderDoesNotExist'
```

Topic: Creating New Folders with os.makedirs()

05/40 05_creating_new_folders.py [shell]

Interactive: type each plain line into IDLE; # OUT: lines are expected results.

Interactive session from the book. Run line-by-line in IDLE.

Lines starting with >>> are input; other lines are expected output (commented).

```
import os
os.makedirs('C:\\delicious\\walnut\\waffles')
```

Topic: Handling Absolute and Relative Paths

06/40 06_handling_absolute_relative.py [shell]

Interactive: type each plain line into IDLE; # OUT: lines are expected results.

Interactive session from the book. Run line-by-line in IDLE.

Lines starting with >>> are input; other lines are expected output (commented).

```
os.path.abspath('.')
```

```
# OUT: 'C:\\Python34'
```

```
os.path.abspath('..\\Scripts')
```

```
# OUT: 'C:\\Python34\\Scripts'
```

```
os.path.isabs('.')
```

```
# OUT: False
```

```
os.path.isabs(os.path.abspath('.'))
```

```
# OUT: True
```

07/40 07_handling_absolute_relative_os.py [shell]

Interactive: type each plain line into IDLE; # OUT: lines are expected results.

```
# Interactive session from the book. Run line-by-line in IDLE.  
# Lines starting with >>> are input; other lines are expected output (commented).  
  
os.path.relpath('C:\\Windows', 'C:\\')  
# OUT: 'Windows'  
os.path.relpath('C:\\Windows', 'C:\\spam\\eggs')  
# OUT: '..\\..\\Windows'  
os.getcwd() 'C:\\Python34'
```

08/40 08_handling_absolute_relative_path.py [shell]

Interactive: type each plain line into IDLE; # OUT: lines are expected results.

```
# Interactive session from the book. Run line-by-line in IDLE.  
# Lines starting with >>> are input; other lines are expected output (commented).  
  
path = 'C:\\Windows\\System32\\calc.exe'  
os.path.basename(path)  
# OUT: 'calc.exe'  
os.path.dirname(path)  
# OUT: 'C:\\Windows\\System32'
```

09/40 09_handling_absolute_relative_calcfilepath.py [shell]

Interactive: type each plain line into IDLE; # OUT: lines are expected results.

```
# Interactive session from the book. Run line-by-line in IDLE.
```

```
# Lines starting with >>> are input; other lines are expected output (commented).
```

```
calcFilePath = 'C:\\Windows\\System32\\calc.exe'
```

```
os.path.split(calcFilePath)
```

```
# OUT: ('C:\\Windows\\System32', 'calc.exe')
```

10/40 10_handling_absolute_relative_os_2.py [shell]

Interactive: type each plain line into IDLE; # OUT: lines are expected results.

```
# Interactive session from the book. Run line-by-line in IDLE.  
# Lines starting with >>> are input; other lines are expected output (commented).  
  
(os.path.dirname(calcFilePath), os.path.basename(calcFilePath))  
# OUT: ('C:\\Windows\\System32', 'calc.exe')
```

11/40 11_handling_absolute_relative_calcfilepath_2.py [shell]

Interactive: type each plain line into IDLE; # OUT: lines are expected results.

```
# Interactive session from the book. Run line-by-line in IDLE.
```

```
# Lines starting with >>> are input; other lines are expected output (commented).
```

```
calcFilePath.split(os.path.sep)
```

```
# OUT: ['C:', 'Windows', 'System32', 'calc.exe']
```

12/40 12_handling_absolute_relative_usr.py [shell]

Interactive: type each plain line into IDLE; # OUT: lines are expected results.

```
# Interactive session from the book. Run line-by-line in IDLE.
```

```
# Lines starting with >>> are input; other lines are expected output (commented).
```

```
'/usr/bin'.split(os.path.sep)
```

```
# OUT: ['', 'usr', 'bin']
```

Topic: Finding File Sizes and Folder Contents

13/40 13_finding_file_sizes.py [shell]

Interactive: type each plain line into IDLE; # OUT: lines are expected results.

Interactive session from the book. Run line-by-line in IDLE.

Lines starting with >>> are input; other lines are expected output (commented).

```
os.path.getsize('C:\\Windows\\System32\\calc.exe')
```

```
# OUT: 776192
```

```
os.listdir('C:\\Windows\\System32')
```

```
# OUT: ['0409', '12520437.cpx', '12520850.cpx', '5U877.ax', 'aaclient.dll',
```

```
# OUT: --snip--
```

```
# OUT: 'xwtpdui.dll', 'xwtpw32.dll', 'zh-CN', 'zh-HK', 'zh-TW', 'zipfldr.dll']
```

14/40 14_finding_file_sizes_totalsize.py [shell]

Interactive: type each plain line into IDLE; # OUT: lines are expected results.

```
# Interactive session from the book. Run line-by-line in IDLE.  
# Lines starting with >>> are input; other lines are expected output (commented).  
  
totalSize = 0  
for filename in os.listdir('C:\\Windows\\System32'):  
# OUT:      totalSize = totalSize + os.path.getsize(os.path.join('C:\\Windows\\System32', filename))  
  
print(totalSize)  
# OUT: 1117846456
```

Topic: Checking Path Validity

15/40 15_checking_path_validity.py [shell]

Interactive: type each plain line into IDLE; # OUT: lines are expected results.

Interactive session from the book. Run line-by-line in IDLE.

Lines starting with >>> are input; other lines are expected output (commented).

```
os.path.exists('C:\\Windows')
# OUT: True
os.path.exists('C:\\some_made_up_folder')
# OUT: False
os.path.isdir('C:\\Windows\\System32')
# OUT: True
os.path.isfile('C:\\Windows\\System32')
# OUT: False
os.path.isdir('C:\\Windows\\System32\\calc.exe')
# OUT: False
os.path.isfile('C:\\Windows\\System32\\calc.exe')
# OUT: True
```

16/40 16_checking_path_validity_os.py [shell]

Interactive: type each plain line into IDLE; # OUT: lines are expected results.

Interactive session from the book. Run line-by-line in IDLE.

Lines starting with >>> are input; other lines are expected output (commented).

```
os.path.exists('D:\\')
```

```
# OUT: False
```

Topic: Opening Files with the open() Function

17/40 17_opening_files_open.py [shell]

Interactive: type each plain line into IDLE; # OUT: lines are expected results.

Interactive session from the book. Run line-by-line in IDLE.

Lines starting with >>> are input; other lines are expected output (commented).

```
helloFile = open('C:\\Users\\your_home_folder\\hello.txt')
```

18/40 18_opening_files_open_hellofile.py [shell]

Interactive: type each plain line into IDLE; # OUT: lines are expected results.

```
# Interactive session from the book. Run line-by-line in IDLE.
```

```
# Lines starting with >>> are input; other lines are expected output (commented).
```

```
helloFile = open('/Users/your_home_folder/hello.txt')
```

Topic: Reading the Contents of Files

19/40 19_reading_contents_files.py [shell]

Interactive: type each plain line into IDLE; # OUT: lines are expected results.

Interactive session from the book. Run line-by-line in IDLE.

Lines starting with >>> are input; other lines are expected output (commented).

```
helloContent = helloFile.read()
```

```
helloContent
```

```
# OUT: 'Hello world!'
```

20/40 20_reading_contents_files_when.py [script]

Script: save and run; predict the output first, then compare.

```
When, in disgrace with fortune and men's eyes,  
I all alone beweep my outcast state,  
And trouble deaf heaven with my bootless cries,  
And look upon myself and curse my fate,
```

21/40 21_reading_contents_files_sonnetfile.py [shell]

Interactive: type each plain line into IDLE; # OUT: lines are expected results.

```
# Interactive session from the book. Run line-by-line in IDLE.  
# Lines starting with >>> are input; other lines are expected output (commented).  
  
sonnetFile = open('sonnet29.txt')  
sonnetFile.readlines()  
# OUT: [When, in disgrace with fortune and men's eyes,\n', ' I all alone beweeep my  
# OUT: outcast state,\n', And trouble deaf heaven with my bootless cries,\n', And  
# OUT: look upon myself and curse my fate,']
```

Topic: Writing to Files

22/40 22_writing_files.py [shell]

Interactive: type each plain line into IDLE; # OUT: lines are expected results.

Interactive session from the book. Run line-by-line in IDLE.

Lines starting with >>> are input; other lines are expected output (commented).

```
baconFile = open('bacon.txt', 'w')
baconFile.write('Hello world!\n')
# OUT: 13
baconFile.close()
baconFile = open('bacon.txt', 'a')
baconFile.write('Bacon is not a vegetable.')
# OUT: 25
baconFile.close()
baconFile = open('bacon.txt')
content = baconFile.read()
baconFile.close()
print(content)
# OUT: Hello world!
# OUT: Bacon is not a vegetable.
```

Topic: Saving Variables with the shelve Module

23/40 23_saving_variables_shelve.py [shell]

Interactive: type each plain line into IDLE; # OUT: lines are expected results.

Interactive session from the book. Run line-by-line in IDLE.

Lines starting with >>> are input; other lines are expected output (commented).

```
import shelve
shelfFile = shelve.open('mydata')
cats = ['Zophie', 'Pooka', 'Simon']
shelfFile['cats'] = cats
shelfFile.close()
```

24/40 24_saving_variables_shelve_shelffile.py [shell]

Interactive: type each plain line into IDLE; # OUT: lines are expected results.

```
# Interactive session from the book. Run line-by-line in IDLE.  
# Lines starting with >>> are input; other lines are expected output (commented).  
  
shelfFile = shelve.open('mydata')  
type(shelfFile)  
# OUT: <class 'shelve.DbfilenameShelf'>  
shelfFile['cats']  
# OUT: ['Zophie', 'Pooka', 'Simon']  
shelfFile.close()
```

25/40 25_saving_variables_shelve_shelffile_2.py [shell]

Interactive: type each plain line into IDLE; # OUT: lines are expected results.

```
# Interactive session from the book. Run line-by-line in IDLE.  
# Lines starting with >>> are input; other lines are expected output (commented).  
  
shelfFile = shelve.open('mydata')  
list(shelfFile.keys())  
# OUT: ['cats']  
list(shelfFile.values())  
# OUT: [['Zophie', 'Pooka', 'Simon']]  
shelfFile.close()
```

Topic: Saving Variables with the pprint.pformat() Function

26/40 26_saving_variables_pprint.py [shell]

Interactive: type each plain line into IDLE; # OUT: lines are expected results.

```
# Interactive session from the book. Run line-by-line in IDLE.  
# Lines starting with >>> are input; other lines are expected output (commented).  
  
>>> import pprint  
>>> cats = [{'name': 'Zophie', 'desc': 'chubby'}, {'name': 'Pooka', 'desc': 'fluffy'}]  
>>> pprint.pformat(cats)  
# OUT: "[{'desc': 'chubby', 'name': 'Zophie'}, {'desc': 'fluffy', 'name': 'Pooka'}]"  
>>> fileObj = open('myCats.py', 'w')  
>>> fileObj.write('cats = ' + pprint.pformat(cats) + '\n')  
# OUT: 83  
>>> fileObj.close()
```

27/40 27_saving_variables_pprint_mycats.py [shell]

Interactive: type each plain line into IDLE; # OUT: lines are expected results.

```
# Interactive session from the book. Run line-by-line in IDLE.  
# Lines starting with >>> are input; other lines are expected output (commented).  
  
import myCats  
myCats.cats  
# OUT: [{'name': 'Zophie', 'desc': 'chubby'}, {'name': 'Pooka', 'desc': 'fluffy'}]  
myCats.cats[0]  
# OUT: {'name': 'Zophie', 'desc': 'chubby'}  
myCats.cats[0]['name']  
# OUT: 'Zophie'
```

Topic: Step 1: Store the Quiz Data in a Dictionary

28/40 28_step_1_store.py [script]

Script: save and run; predict the output first, then compare.

```
#!/ python3
# randomQuizGenerator.py - Creates quizzes with questions and answers in
# random order, along with the answer key.

import random

# The quiz data. Keys are states and values are their capitals.
capitals = {'Alabama': 'Montgomery', 'Alaska': 'Juneau', 'Arizona': 'Phoenix',
            'Arkansas': 'Little Rock', 'California': 'Sacramento', 'Colorado': 'Denver',
            'Connecticut': 'Hartford', 'Delaware': 'Dover', 'Florida': 'Tallahassee',
            'Georgia': 'Atlanta', 'Hawaii': 'Honolulu', 'Idaho': 'Boise', 'Illinois':
            'Springfield', 'Indiana': 'Indianapolis', 'Iowa': 'Des Moines', 'Kansas':
            'Topeka', 'Kentucky': 'Frankfort', 'Louisiana': 'Baton Rouge', 'Maine':
            'Augusta', 'Maryland': 'Annapolis', 'Massachusetts': 'Boston', 'Michigan':
            'Lansing', 'Minnesota': 'Saint Paul', 'Mississippi': 'Jackson', 'Missouri':
            'Jefferson City', 'Montana': 'Helena', 'Nebraska': 'Lincoln', 'Nevada':
            'Carson City', 'New Hampshire': 'Concord', 'New Jersey': 'Trenton', 'New
            Mexico': 'Santa Fe', 'New York': 'Albany', 'North Carolina': 'Raleigh',
            'North Dakota': 'Bismarck', 'Ohio': 'Columbus', 'Oklahoma': 'Oklahoma City',
            'Oregon': 'Salem', 'Pennsylvania': 'Harrisburg', 'Rhode Island': 'Providence',
            'South Carolina': 'Columbia', 'South Dakota': 'Pierre', 'Tennessee':
            'Nashville', 'Texas': 'Austin', 'Utah': 'Salt Lake City', 'Vermont':
            'Montpelier', 'Virginia': 'Richmond', 'Washington': 'Olympia', 'West
            Virginia': 'Charleston', 'Wisconsin': 'Madison', 'Wyoming': 'Cheyenne'}

# Generate 35 quiz files.
for quizNum in range(35):
    # TODO: Create the quiz and answer key files.

    # TODO: Write out the header for the quiz.

    # TODO: Shuffle the order of the states.

    # TODO: Loop through all 50 states, making a question for each.
```

Topic: Step 2: Create the Quiz File and Shuffle the Question Order

29/40 29_step_2_create.py [script]

Script: save and run; predict the output first, then compare.

```
#!/ python3
# randomQuizGenerator.py - Creates quizzes with questions and answers in
# random order, along with the answer key.

--snip--

# Generate 35 quiz files.
for quizNum in range(35):
    # Create the quiz and answer key files.
    quizFile = open('capitalsquiz%s.txt' % (quizNum + 1), 'w')
    answerKeyFile = open('capitalsquiz_answers%s.txt' % (quizNum + 1), 'w')

    # Write out the header for the quiz.
    quizFile.write('Name:\n\nDate:\n\nPeriod:\n\n')
    quizFile.write((' ' * 20) + 'State Capitals Quiz (Form %s)' % (quizNum + 1))
    quizFile.write('\n\n')

    # Shuffle the order of the states.
    states = list(capitals.keys())
    random.shuffle(states)

    # TODO: Loop through all 50 states, making a question for each.
```

Topic: Step 3: Create the Answer Options

30/40 30_step_3_create.py [script]

Script: save and run; predict the output first, then compare.

```
#!/ python3
# randomQuizGenerator.py - Creates quizzes with questions and answers in
# random order, along with the answer key.

--snip--

# Loop through all 50 states, making a question for each.
for questionNum in range(50):

    # Get right and wrong answers.
    correctAnswer = capitals[states[questionNum]]
    wrongAnswers = list(capitals.values())
    del wrongAnswers[wrongAnswers.index(correctAnswer)]
    wrongAnswers = random.sample(wrongAnswers, 3)
    answerOptions = wrongAnswers + [correctAnswer]
    random.shuffle(answerOptions)

    # TODO: Write the question and answer options to the quiz file.

    # TODO: Write the answer key to a file.
```

Topic: Step 4: Write Content to the Quiz and Answer Key Files

31/40 31_step_4_write.py [script]

Script: save and run; predict the output first, then compare.

```
#!/ python3
# randomQuizGenerator.py - Creates quizzes with questions and answers in
# random order, along with the answer key.

--snip--
# Loop through all 50 states, making a question for each.
for questionNum in range(50):
    --snip--

    # Write the question and the answer options to the quiz file.
    quizFile.write('%s. What is the capital of %s?\n' % (questionNum + 1,
        states[questionNum]))
    for i in range(4):
        quizFile.write(' %s. %s\n' % ('ABCD'[i], answerOptions[i]))
    quizFile.write('\n')

    # Write the answer key to a file.
    answerKeyFile.write('%s. %s\n' % (questionNum + 1, 'ABCD'[
        answerOptions.index(correctAnswer)]))
quizFile.close()
answerKeyFile.close()
```

32/40 32_step_4_write_name.py [script]

Script: save and run; predict the output first, then compare.

Name:

Date:

Period:

State Capitals Quiz (Form 1)

1. What is the capital of West Virginia?

- A. Hartford
- B. Santa Fe
- C. Harrisburg
- D. Charleston

2. What is the capital of Colorado?

- A. Raleigh
- B. Harrisburg
- C. Denver
- D. Lincoln

--snip--

33/40 33_step_4_write_d.py [script]

Script: save and run; predict the output first, then compare.

```
1. D
2. C
3. A
4. C
--snip--
```

Topic: Project: Multiclipboard

34/40 34_project_multiclipboard.py [script]

Script: save and run; predict the output first, then compare.

```
@pyw.exe C:\Python34\mcb.pyw %*
```

Topic: Step 1: Comments and Shelf Setup

35/40 35_step_1_comments.py [script]

Script: save and run; predict the output first, then compare.

```
#!/ python3
# mcb.pyw - Saves and loads pieces of text to the clipboard.
# Usage: py.exe mcb.pyw save <keyword> - Saves clipboard to keyword.
#       py.exe mcb.pyw <keyword> - Loads keyword to clipboard.
#       py.exe mcb.pyw list - Loads all keywords to clipboard.

import shelve, pyperclip, sys

mcbShelf = shelve.open('mcb')

# TODO: Save clipboard content.

# TODO: List keywords and load content.

mcbShelf.close()
```

Topic: Step 2: Save Clipboard Content with a Keyword

36/40 36_step_2_save.py [script]

Script: save and run; predict the output first, then compare.

```
#!/ python3
# mcb.pyw - Saves and loads pieces of text to the clipboard.
--snip--

# Save clipboard content.
if len(sys.argv) == 3 and sys.argv[1].lower() == 'save':
    mcbShelf[sys.argv[2]] = pyperclip.paste()
elif len(sys.argv) == 2:
    # TODO: List keywords and load content.

mcbShelf.close()
```

Topic: Step 3: List Keywords and Load a Keyword's Content

37/40 37_step_3_list.py [script]

Script: save and run; predict the output first, then compare.

```
#!/ python3
# mcb.pyw - Saves and loads pieces of text to the clipboard.
--snip--

# Save clipboard content.
if len(sys.argv) == 3 and sys.argv[1].lower() == 'save':
    mcbShelf[sys.argv[2]] = pyperclip.paste()
elif len(sys.argv) == 2:
    # List keywords and load content.
    if sys.argv[1].lower() == 'list':
        pyperclip.copy(str(list(mcbShelf.keys())))
    elif sys.argv[1] in mcbShelf:
        pyperclip.copy(mcbShelf[sys.argv[1]])

mcbShelf.close()
```

Topic: Mad Libs

38/40 38_mad_libs.py [script]

Script: save and run; predict the output first, then compare.

The ADJECTIVE panda walked to the NOUN and then VERB. A nearby NOUN was unaffected by these events.

39/40 39_mad_libs_enter.py [script]

Script: save and run; predict the output first, then compare.

Enter an adjective:

silly

Enter a noun:

chandelier

Enter a verb:

screamed

Enter a noun:

pickup truck

40/40 40_mad_libs_the.py [script]

Script: save and run; predict the output first, then compare.

```
The silly panda walked to the chandelier and then screamed. A nearby pickup
truck was unaffected by these events.
```